

CSE 422: Computer Graphics Lab

Lab 03: Implementation of DDA Line Drawing Algorithm Using OpenGL

Lab Objective

After completing this lab, students will be able to:

- Understand the concept of rasterizing a line.
- Implement the DDA (Digital Differential Analyzer) Line Drawing Algorithm.
- Plot points using OpenGL.
- Draw a line between two user-defined endpoints.
- Analyze how slope affects line generation.

Background

Computer screens are made of pixels. A mathematical line is continuous, but a computer display is discrete.

The DDA (Digital Differential Analyzer) algorithm is an incremental scan conversion algorithm used to determine which pixels should be illuminated to represent a line.

For a line between:

(x_1, y_1)

and

(x_2, y_2)

Calculate:

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

The slope is:

$$m = dy / dx$$

Depending on the slope, either x or y is incremented and the corresponding coordinate is calculated.

DDA Algorithm (General Form)

Step 1:

Calculate

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

Step 2:

Determine

$$\text{steps} = \max(|dx|, |dy|)$$

Step 3:

Calculate increments

$$x\text{Increment} = dx / \text{steps}$$

$$y\text{Increment} = dy / \text{steps}$$

Step 4:

Initialize

$$x = x_1$$

$$y = y_1$$

Step 5:

Repeat for each step

Plot(round(x), round(y))

$$x = x + x\text{Increment}$$

$$y = y + y\text{Increment}$$

Relationship Between Classical DDA and General DDA

The DDA (Digital Differential Analyzer) algorithm is traditionally explained using two separate cases based on the slope of the line.

Classical DDA Approach

For a line between two endpoints:

(x_1, y_1) and (x_2, y_2)

the slope is calculated as:

$$m = (y_2 - y_1)/(x_2 - x_1)$$

Case 1: $|m| \leq 1$

When the slope is less than or equal to 1, the line changes more in the x-direction than in the y-direction. Therefore, x is incremented by 1 in each step and the corresponding y value is calculated.

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$

Case 2: $|m| > 1$

When the slope is greater than 1, the line changes more in the y-direction than in the x-direction. Therefore, y is incremented by 1 in each step and the corresponding x value is calculated.

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + (1/m)$$

This approach helps students understand the fundamental idea of DDA: always advance along the dominant direction and calculate the other coordinate incrementally.

Limitation of the Classical Approach

The classical formulation requires separate handling for:

- Slope less than 1
- Slope greater than 1
- Negative slopes
- Vertical lines
- Reverse directions

As a result, the implementation becomes longer and more complex.

General DDA Approach

The general DDA algorithm extends the same idea by automatically selecting the dominant direction.

First calculate:

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

Then determine the number of steps as:

$$\text{steps} = \max(|dx|, |dy|)$$

The coordinate increments become:

$$x\text{Increment} = dx / \text{steps}$$

$$y\text{Increment} = dy / \text{steps}$$

Starting from (x_1, y_1) , the next point is generated using:

$$x = x + x\text{Increment}$$

$$y = y + y\text{Increment}$$

for each step.

Why Does General DDA Work?

The value of "steps" is chosen as the larger of $|dx|$ and $|dy|$.

- If $|dx| > |dy|$, then $x\text{Increment}$ is approximately ± 1 and the algorithm behaves like the classical case where x is incremented at every step.
- If $|dy| > |dx|$, then $y\text{Increment}$ is approximately ± 1 and the algorithm behaves like the classical case where y is incremented at every step.

Therefore, the general DDA algorithm automatically handles both classical cases without requiring separate conditions based on the slope.

Key Observation

The classical DDA and general DDA are based on the same principle:

"Move one step at a time along the dominant axis and calculate the corresponding coordinate incrementally."

The general DDA formulation is preferred in programming because it is shorter, easier to implement, and works for lines of any slope and direction.

OpenGL Program Structure

Students must complete the missing portions marked with TODO.

```
#include <GL/gl.h>
#include <GL/glut.h>
#include <stdio.h>
#include <math.h>

// Global Variables

float x1, y_1, x2, y2;

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0, 1.0, 0.0);

    glBegin(GL_POINTS);

    // =====
    // TODO:
    // Implement DDA Algorithm
    // Plot point using: glVertex2f(x, y)
    // =====

    glEnd();

    glFlush();
}

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);

    glMatrixMode(GL_PROJECTION);
```

```

    glLoadIdentity();

    glOrtho(0, 100, 0, 100, -1, 1);
}

int main(int argc, char** argv)
{
    printf("Enter First Point (x1 y_1): ");
    scanf("%f %f", &x1, &y_1);

    printf("Enter Second Point (x2 y2): ");
    scanf("%f %f", &x2, &y2);

    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    glutInitWindowSize(600, 600);

    glutInitWindowPosition(100, 100);

    glutCreateWindow("DDA Line Drawing");

    init();

    glutDisplayFunc(display);

    glutMainLoop();

    return 0;
}

```

OpenGL Functions Used

Function	Purpose
glClear()	Clears the screen
glColor3f()	Sets drawing color

<code>glBegin(GL_POINTS)</code>	Starts point drawing mode
<code>glVertex2f()</code>	Plots a point
<code>glEnd()</code>	Ends drawing block
<code>glFlush()</code>	Displays rendered output
<code>glOrtho()</code>	Defines coordinate system

Student Tasks

Task 1

Complete the DDA implementation inside `display()`.

Task 2

Test the program using:

(10,10) to (80,50)

Task 3

Test the program using:

(20,20) to (40,90)

Observe how the slope affects plotting.

Task 4

Test the program using:

(80,20) to (10,70)

Observe whether your implementation handles reverse directions correctly.

Discussion Questions

1. What is the purpose of the DDA algorithm?
2. Why does DDA use floating-point arithmetic?
3. What happens when the slope is greater than 1?
4. What is the role of the variable "steps"?
5. Why are OpenGL points used instead of directly drawing a line?
6. What are the limitations of DDA?
7. Why is Bresenham's algorithm generally faster than DDA?

Expected Output

The program should display a straight line connecting the two user-specified endpoints using individually plotted points generated by the DDA algorithm.

Submission Requirements

Students must submit:

1. Source Code (.cpp)
2. Screenshot of Output
3. Brief explanation of the DDA algorithm
4. Answers to Discussion Questions

Lab Report Format:

Title: Design and Implementation of a 2D House using OpenGL

Introduction: Explain what you have done in this project.

Contents: Which functions and shapes you have used; describe them briefly.

Graph: Draw your project output by hand.

Code: Full code

Output: Screenshot of output

Discussion: Briefly describe the task and how you solved it.

Solve:

☒ Task 1

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0, 1.0, 0.0);

    glBegin(GL_POINTS);

    float dx = x2 - x1;
    float dy = y2 - y_1;

    float steps;

    if (fabs(dx) > fabs(dy))
        steps = fabs(dx);
    else
        steps = fabs(dy);

    float Xinc = dx / steps;
    float Yinc = dy / steps;

    float x = x1;
    float y = y_1;

    for (int i = 0; i <= steps; i++)
    {
        glVertex2f(round(x), round(y));

        x += Xinc;
        y += Yinc;
    }

    glEnd();

    glFlush();
}
```

☑ Task 2

Input:

$(10,10) \rightarrow (80,50)$

Observation:

- $dx = 70, dy = 40$
- $\text{slope} < 1$ (flat line type)
- x বেশি change করছে, তাই line horizontally বেশি spread হবে
- DDA x-direction এ বেশি advance করবে

Result:

- ✓ একটি সুন্দর diagonal line তৈরি হবে (left-bottom $\rightarrow$ right-top)
 - ✓ points বেশি spread হবে x-axis বরাবর
-

☑ Task 3

Input:

$(20,20) \rightarrow (40,90)$

Observation:

- $dx = 20, dy = 70$
- $\text{slope} > 1$ (steep line)
- y বেশি change করছে

Result:

- ✓ line vertically বেশি উঠবে
 - ✓ points বেশি ঘন হবে y-axis বরাবর
 - ✓ line দেখতে “almost vertical” লাগবে
-

☑ Task 4

Input:

(80,20) → (10,70)

Observation:

- $dx = -70$, $dy = 50$
- reverse direction (right → left)
- negative slope handling test

Result:

- ✓ line ঠিকভাবে draw হবে (right-top → left-bottom direction)
 - ✓ তোমার DDA যদি correct হয়, তাহলে direction কোনো সমস্যা করবে না
 - ✓ negative dx, dy properly handle হবে $steps = \max(|dx|, |dy|)$ এর কারণে
-

★ Final Key Understanding (Report এ লিখতে পারো)

- Task 2 → normal slope (<1), horizontal movement বেশি
- Task 3 → steep slope (>1), vertical movement বেশি
- Task 4 → reverse direction, negative dx handle হয়
- General DDA works for all cases because it uses $steps = \max(|dx|, |dy|)$

Discussion Questions Answers

1. What is the purpose of the DDA algorithm?

DDA algorithm is used to generate the pixels required to draw a straight line between two points on a raster display.

2. Why does DDA use floating-point arithmetic?

Because the coordinate increments are often fractional values, floating-point arithmetic is needed for accurate calculations.

3. What happens when the slope is greater than 1?

The change in y is greater than the change in x, so the algorithm effectively advances mainly along the y-direction.

4. What is the role of the variable "steps"?

`steps` determines how many points will be plotted and ensures smooth line generation by choosing the larger of $|dx|$ and $|dy|$.

5. Why are OpenGL points used instead of directly drawing a line?

To demonstrate how the DDA algorithm generates individual pixels that form a line.

6. What are the limitations of DDA?

It uses floating-point calculations, which make it slower and can introduce rounding errors.

7. Why is Bresenham's algorithm generally faster than DDA?

Because Bresenham uses only integer arithmetic and avoids floating-point operations.

Brief Explanation of DDA Algorithm

The Digital Differential Analyzer (DDA) is a line drawing algorithm used in computer graphics. It calculates the difference between the starting and ending coordinates, determines the number of steps required, and incrementally generates intermediate points. Each calculated point is plotted on the screen, producing a straight line between the two endpoints. DDA is simple to implement and works for lines of any slope.